C++ STL Notes

A Practical Guide for Students

These notes cover the Standard Template Library (STL) in C++. They focus on the concepts and operations you will use most often in coding practice, interviews, and projects.

---

1. What Is the STL?

The **Standard Template Library (STL)** is a collection of ready-made C++ classes and functions that help us store, access, process, and organize data.

Instead of implementing common data structures and algorithms from scratch, we can use tested library components.

For example, instead of manually creating a resizable array, use `vector`.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> numbers = {10, 20, 30};

    numbers.push_back(40);

    for (int number : numbers) {
        cout << number << " ";
    }
}
```

Output:

```
10 20 30 40
```

Main parts of the STL

Part	Purpose	Examples
Containers	Store data	`vector`, `list`, `map`, `set`
Iterators	Refer to elements in containers	`begin()`, `end()`
Algorithms	Perform operations on data	`sort()`, `find()`, `reverse()`
Function objects / callables	Customize operations	Comparators, lambdas

> Note: Containers and algorithms are designed to work together through iterators.

---

2. Headers You Will Commonly Need

Include the header that declares the feature you use.

Header	Common features
`<vector>`	`vector`
`<array>`	`array`
`<list>`	`list`
`<deque>`	`deque`
`<stack>`	`stack`
`<queue>`	`queue`, `priority_queue`
`<map>`	`map`, `multimap`
`<unordered_map>`	`unordered_map`
`<set>`	`set`, `multiset`
`<unordered_set>`	`unordered_set`
`<algorithm>`	`sort`, `find`, `reverse`, `unique`, etc.
`<numeric>`	`accumulate`
`<iterator>`	Iterator utilities such as `distance`
`<string>`	`string`
`<utility>`	`pair`, `move`, and related utilities

For small demonstrations, `using namespace std;` is used for readability. In larger projects, many developers prefer writing `std::vector`, `std::sort`, and so on.

---

3. Iterators

An **iterator** is an object used to refer to an element in a container. It behaves somewhat like a pointer.

- `begin()` refers to the first element.
- `end()` refers to the position **after** the last element. It is not a valid element to dereference.
- `cbegin()` and `cend()` provide read-only iterators.
- `rbegin()` and `rend()` iterate in reverse for containers that support them.

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int main() {
    vector<int> values = {10, 20, 30};

    for (auto it = values.begin(); it != values.end(); ++it) {
        cout << *it << " ";
    }
}
```

Output:

```
10 20 30
```

Here, `*it` accesses the element referred to by the iterator.

`auto`

`auto` lets the compiler infer a variable's type.

```
auto number = 10;       // int
auto price = 99.5;      // double
auto it = values.begin();
```

Use `const auto&` when you want to read each element without copying it:

```
for (const auto& value : values) {
    cout << value << " ";
}
```

Range-based `for` loop

For simple traversal, this is usually the clearest option:

```
for (int value : values) {
    cout << value << " ";
}
```

Use `int& value` if you need to modify the original elements:

```
for (int& value : values) {
    value *= 2;
}
```

---

4. Sequence Containers

Sequence containers store elements in a sequence.

4.1 `vector`

A `vector` is a dynamic array. It can grow or shrink as elements are added or removed.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {10, 20, 30};

    nums.push_back(40);   // Add at the end
    nums.pop_back();      // Remove the last element

    cout << nums[0] << "\n";       // 10
    cout << nums.at(1) << "\n";    // 20
    cout << nums.size() << "\n";   // 3
    cout << nums.empty() << "\n";  // 0 (false)
}
```

Common operations:

Operation	Meaning
`push_back(x)`	Add `x` at the end
`pop_back()`	Remove the last element
`size()`	Number of elements
`empty()`	Check whether there are no elements
`front()` / `back()`	First / last element
`at(i)`	Access index with bounds checking
`clear()`	Remove all elements
`insert(pos, x)`	Insert before iterator `pos`
`erase(pos)`	Remove the element at iterator `pos`

Operation	Meaning
`resize(n)`	Change the number of elements
`capacity()`	Current allocated storage capacity

Important:

- `nums[i]` does not perform bounds checking; `nums.at(i)` does.
- `size()` is the number of elements; `capacity()` is how many elements can fit in the current allocation before another allocation is needed.
- `pop_back()` should only be used when the vector is not empty.
- Inserting or erasing can invalidate iterators and references at or after the affected position. Growing a vector can invalidate all iterators and references if reallocation occurs.

Insert and erase example:

```
vector<int> nums = {10, 20, 30, 40};

nums.insert(nums.begin() + 1, 15); // {10, 15, 20, 30, 40}
nums.erase(nums.begin() + 2);      // {10, 15, 30, 40}
```

Use `vector` as the default container for many array-like tasks.

4.2 `array`

`std::array` is a fixed-size array that provides STL-style methods.

```
#include <array>
array<int, 3> values = {10, 20, 30};
cout << values.size();
```

The size is fixed at compile time.

4.3 `list`

A `list` is a doubly linked list.

```
#include <list>
list<int> values = {10, 20, 30};

values.push_front(5);
values.push_back(40);
values.pop_front();
```

- Efficient insertion and deletion when you already have an iterator to the position.

- No fast random access like `values[2]`.
- Searching for a position is generally linear.

4.4 `deque`

A `deque` (double-ended queue) supports efficient insertion and removal at both ends.

```
#include <deque>
deque<int> values = {10, 20, 30};

values.push_front(5);
values.push_back(40);
values.pop_front();
values.pop_back();
```

It supports indexed access, but its storage is not one single contiguous array like a vector.

---

5. Container Adapters

Container adapters provide a restricted interface for a particular way of processing data.

5.1 `stack` — LIFO

LIFO means Last In, First Out. The last item added is the first item removed.

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> st;

    st.push(10);
    st.push(20);
    st.push(30);

    cout << st.top() << "\n"; // 30

    st.pop();                 // Removes 30
    cout << st.top() << "\n"; // 20
}
```

Common methods: `push()`, `pop()`, `top()`, `empty()`, `size()`.

`pop()` removes an element but does not return it.

5.2 `queue` — FIFO

FIFO means First In, First Out. The first item added is the first item removed.

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<int> q;

    q.push(10);
    q.push(20);
    q.push(30);

    cout << q.front() << "\n"; // 10
    q.pop();
    cout << q.front() << "\n"; // 20
}
```

Common methods: `push()`, `pop()`, `front()`, `back()`, `empty()`, `size()`.

5.3 `priority_queue`

A `priority_queue` returns the highest-priority element first. By default, it is a **max-heap**, so the largest value is at the top.

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    priority_queue<int> pq;

    pq.push(30);
    pq.push(10);
    pq.push(50);

    cout << pq.top(); // 50
}
```

A min-heap can be created using `greater<int>`:

```
priority_queue<int, vector<int>, greater<int>> minHeap;
minHeap.push(30);
minHeap.push(10);
minHeap.push(50);

cout << minHeap.top(); // 10
```

Common methods: `push()`, `pop()`, `top()`, `empty()`, `size()`.

---

6. `pair`

A `pair` stores two values, which may have different types.

```
#include <iostream>
#include <utility>
using namespace std;

int main() {
    pair<string, int> student = {"Aarav", 85};

    cout << student.first << "\n";  // Aarav
    cout << student.second << "\n"; // 85
}
```

Pairs are often used to store related values, such as a name and score or a value and its frequency.

A vector of pairs:

```
vector<pair<string, int>> students = {
    {"Aarav", 85},
    {"Diya", 92},
    {"Kabir", 78}
};
```

With the default ordering, pairs compare first by `.first`, then by `.second` if the first values are equal.

---

7. Associative Containers

These containers organize data for searching and lookup.

7.1 `set`

A `set` stores **unique values in sorted order**.

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> numbers = {30, 10, 20, 10};

    for (int number : numbers) {
        cout << number << " ";
```

```
    }
}
```

Output:

```
10 20 30
```

Common operations:

- `insert(value)`
- `erase(value)`
- `find(value)`
- `count(value)`
- `size()`
- `empty()`

`set` typically provides search, insertion, and deletion in $O(\log n)$ time.

7.2 `multiset`

A `multiset` stores values in sorted order but **allows duplicates**.

```
multiset<int> numbers = {10, 20, 10, 30};
// Contains 10, 10, 20, 30
```

7.3 `map`

A `map` stores **key-value pairs** with unique keys, ordered by key.

```cpp
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<string, int> marks;

    marks["Aarav"] = 85;
    marks["Diya"] = 92;
    marks["Aarav"] = 90; // Updates Aarav's existing value

    cout << marks["Aarav"] << "\n"; // 90

    for (const auto& item : marks) {
        cout << item.first << ": " << item.second << "\n";
    }
}
```

Important:

- Each key appears at most once.
- `marks[key]` inserts a default-initialized mapped value if the key does not exist.
- Use `find()` or `contains()` (C++20) when checking whether a key exists without inserting it.

Example lookup:

```cpp
auto it = marks.find("Diya");

if (it != marks.end()) {
    cout << it->second;
} else {
    cout << "Student not found";
}
```

7.4 `multimap`

A `multimap` is like `map`, but it allows multiple entries with the same key.

7.5 `unordered_set` and `unordered_map`

These use hashing and do not keep elements sorted.

```cpp
#include <unordered_map>
unordered_map<string, int> frequency;
frequency["apple"]++;
frequency["banana"]++;
frequency["apple"]++;
```

- `unordered_set`: unique values, no sorted order.
- `unordered_map`: unique keys, no sorted order.
- Average lookup, insertion, and deletion are typically $O(1)$, but the worst case can be $O(n)$.
- Use `map`/`set` when sorted order is needed; use unordered versions when ordering is not required and hash-based lookup is suitable.

---

8. STL Algorithms

Most common algorithms are declared in `<algorithm>`. They generally operate on iterator ranges written as `[first, last)`: the first iterator is included, but the last is excluded.

8.1 `sort()`

Sort a vector in ascending order:

```
#include <algorithm>
#include <vector>
using namespace std;

vector<int> nums = {40, 10, 30, 20};
sort(nums.begin(), nums.end());
// {10, 20, 30, 40}
```

Descending order:

```
sort(nums.begin(), nums.end(), greater<int>());
// {40, 30, 20, 10}
```

8.2 `reverse()`

```
reverse(nums.begin(), nums.end());
```

Reverses the order of elements in the range.

8.3 `find()`

```
auto it = find(nums.begin(), nums.end(), 30);

if (it != nums.end()) {
    cout << "Found";
} else {
    cout << "Not found";
}
```

For a normal vector, `find()` takes linear time, $O(n)$.

8.4 `count()`

```
int occurrences = count(nums.begin(), nums.end(), 20);
```

Counts how many times a value occurs in the range.

8.5 `min_element()` and `max_element()`

These return iterators to the minimum and maximum elements.

```
auto smallest = min_element(nums.begin(), nums.end());
auto largest = max_element(nums.begin(), nums.end());

if (!nums.empty()) {
    cout << *smallest << " " << *largest;
}
```

Check that the container is not empty before dereferencing the returned iterator.

8.6 `binary_search()`

Checks whether a value exists in a **sorted** range.

```
sort(nums.begin(), nums.end());

bool exists = binary_search(nums.begin(), nums.end(), 30);
```

The range must be sorted according to the same ordering used by the search.

8.7 `lower_bound()` and `upper_bound()`

These are used with a sorted range.

- `lower_bound`: first position whose value is not less than the target.

- `upper_bound`: first position whose value is greater than the target.

```
vector<int> nums = {10, 20, 20, 20, 30, 40};

auto first = lower_bound(nums.begin(), nums.end(), 20);
auto afterLast = upper_bound(nums.begin(), nums.end(), 20);

cout << (afterLast - first); // 3 occurrences of 20
```

This iterator subtraction works here because `vector` has random-access iterators. For general iterator types, use `distance(first, afterLast)`.

8.8 `accumulate()`

`accumulate()` is declared in `<numeric>`.

```
#include <numeric>
vector<int> nums = {10, 20, 30};

int total = accumulate(nums.begin(), nums.end(), 0);
cout << total; // 60
```

The initial value affects the result type. For a potentially large sum, use a suitable type such as `0LL`:

```
long long total = accumulate(nums.begin(), nums.end(), 0LL);
```

8.9 `all_of()`, `any_of()`, and `none_of()`

```
vector<int> nums = {2, 4, 6, 8};

bool allEven = all_of(nums.begin(), nums.end(),
                      [](int x) { return x % 2 == 0; });

bool anyGreaterThanFive = any_of(nums.begin(), nums.end(),
                                 [](int x) { return x > 5; });

bool noneNegative = none_of(nums.begin(), nums.end(),
                            [](int x) { return x < 0; });
```

- `all_of`: every element satisfies the condition.
- `any_of`: at least one element satisfies the condition.
- `none_of`: no element satisfies the condition.

8.10 `replace()`

```
vector<int> nums = {1, 2, 2, 3};
replace(nums.begin(), nums.end(), 2, 9);
// {1, 9, 9, 3}
```

Replaces every matching value in the range.

8.11 `remove()` and the erase-remove pattern

`remove()` does **not** change the container's size. It moves unwanted values toward the end and returns a new logical end iterator.

```
vector<int> nums = {1, 2, 3, 2, 4};

nums.erase(remove(nums.begin(), nums.end(), 2), nums.end());
// {1, 3, 4}
```

This is called the **erase-remove idiom**.

8.12 `unique()`

`unique()` removes consecutive duplicate values from the logical range; it does not resize the container by itself.

```
vector<int> nums = {1, 1, 2, 2, 3, 1};

sort(nums.begin(), nums.end());
nums.erase(unique(nums.begin(), nums.end()), nums.end());
// {1, 2, 3}
```

Sorting first groups equal values together, allowing all duplicates to be removed. If you only want to remove adjacent duplicates, sorting is not required.

---

9. Lambda Expressions and Custom Sorting

A **lambda** is a small unnamed function that can be written where it is needed.

General shape:

```
[](parameters) {
    // body
}
```

Example: sort integers by descending order:

```
sort(nums.begin(), nums.end(),
     [](int a, int b) {
         return a > b;
     });
```

Example: sort student records by marks, highest first:

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Student {
    string name;
    int marks;
};

int main() {
    vector<Student> students = {
        {"Aarav", 85},
        {"Diya", 92},
        {"Kabir", 78}
    };

    sort(students.begin(), students.end(),
         [](const Student& a, const Student& b) {
             return a.marks > b.marks;
         });

    for (const auto& student : students) {
        cout << student.name << " " << student.marks << "\n";
    }
}
```

The comparator should define a consistent strict ordering. Avoid comparators that say an element is less than itself or produce contradictory results.

---

10. Quick Complexity Reference

These are typical complexities; exact guarantees vary by operation and container.

Operation	Typical complexity
`vector` index access	$O(1)$
`vector` search using `find`	$O(n)$

Operation	Typical complexity
`vector` append with `push_back`	Amortized $O(1)$
`vector` insert/erase in the middle	$O(n)$
`list` insert/erase at a known iterator	$O(1)$
`set` / `map` lookup, insert, erase	$O(\log n)$
`unordered_set` / `unordered_map` lookup	Average $O(1)$, worst $O(n)$
`sort` on random-access iterators	$O(n \log n)$
`priority_queue` top	$O(1)$
`priority_queue` push/pop	$O(\log n)$
`stack` / `queue` push/pop	Typically $O(1)$

Choose a container based on the operations you need most, not only on how it looks in code.

---

11. Common Mistakes

1. Dereferencing `end()`:

   ```
   // Wrong: cout << *nums.end();
   ```

   `end()` points after the final element.

2. Calling `top()`, `front()`, `back()`, or `pop()` on an empty container. Check `empty()` first.
3. Assuming `map[key]` only reads a value. If the key is missing, it inserts a new entry.
4. Using `binary_search()`, `lower_bound()`, or `upper_bound()` on unsorted data.
5. Expecting `remove()` or `unique()` to shrink the container. Use `erase()` with the returned iterator.
6. Assuming a `set` keeps duplicates. It does not; use `multiset` if duplicates are required.
7. Assuming `unordered_map` iteration is sorted. It is not.
8. Keeping vector iterators or references after an operation that may invalidate them.
9. Using `nums[i]` with an invalid index. Use valid bounds or `at()` where checked access is desired.
10. Forgetting the correct header, especially `<numeric>` for `accumulate()`.

---

12. Practice Problems

Try solving these independently before looking for solutions.

Beginner

1. Create a `vector<int>` containing five numbers. Print them using a range-based loop.
2. Add three numbers using `push_back()` and remove the last number.
3. Find the size, first element, and last element of a non-empty vector.
4. Sort six numbers in ascending and descending order.
5. Count how many times a given number occurs in a vector.
6. Find the minimum and maximum values in a vector.
7. Reverse a vector.
8. Store three student names and marks using `pair`.

Intermediate

9. Remove duplicate values from a vector.
10. Count the frequency of each number using `map` or `unordered_map`.
11. Count the frequency of each word in a sentence.
12. Use a `set` to print unique numbers in sorted order.
13. Use a `stack` to reverse a sequence of characters.
14. Use a `queue` to simulate people waiting in a line.
15. Find the kth-largest element using `priority_queue`.
16. Sort students by marks from highest to lowest; break ties by name.
17. Find all common elements between two arrays.
18. Use `lower_bound()` and `upper_bound()` to count occurrences in a sorted vector.
19. Remove all occurrences of a chosen value using the erase-remove pattern.
20. Group values by frequency and print the frequency of each value.

Project-level practice

Build a small **Student Record Manager** using STL:

- Store student records in a `vector`.
- Store each student's name and marks in a `struct` or class.
- Add and display students.
- Search for a student by name.
- Sort students by marks.
- Use a `map` to count grades or result categories.
- Later, extend it with file handling to save and load records.

---

13. Recommended Learning Order

Study the topics in this order:

1. `vector`, range-based loops, `auto`, iterators
2. `sort`, `find`, `count`, `reverse`, `min_element`, `max_element`
3. `pair` and `vector<pair<...>>`
4. `stack`, `queue`, `priority_queue`
5. `set`, `map`, `unordered_set`, `unordered_map`
6. `accumulate`, `binary_search`, `lower_bound`, `upper_bound`

7. `remove`, `unique`, erase-remove
8. Lambdas and custom comparators
9. Complexity and iterator invalidation
10. Solve 10–15 problems without copying solutions

You do not need to master every advanced STL feature at the beginning. First learn to choose a suitable container, use its core operations correctly, and solve problems independently.

---

14. Final Revision Checklist

- [ ] I can explain containers, iterators, and algorithms.
- [ ] I can use `vector`, including insertion and deletion.
- [ ] I understand `begin()` and `end()`.
- [ ] I can use `sort`, `find`, `count`, `reverse`, and `accumulate`.
- [ ] I understand when data must be sorted for binary-search algorithms.
- [ ] I know the difference between `set` and `multiset`.
- [ ] I know the difference between `map` and `unordered_map`.
- [ ] I can use `stack`, `queue`, and `priority_queue`.
- [ ] I can use `pair` and a lambda comparator.
- [ ] I understand erase-remove and how `unique()` works.
- [ ] I can explain basic time complexity.
- [ ] I have solved at least 10 STL practice problems independently.